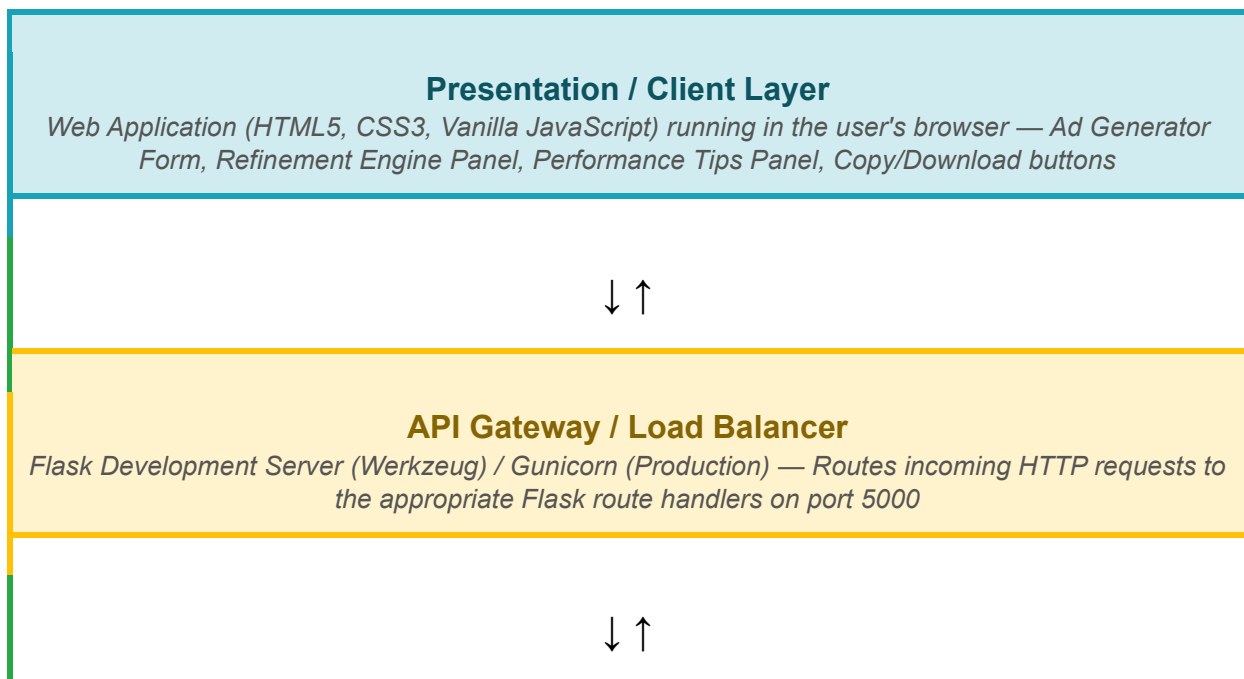


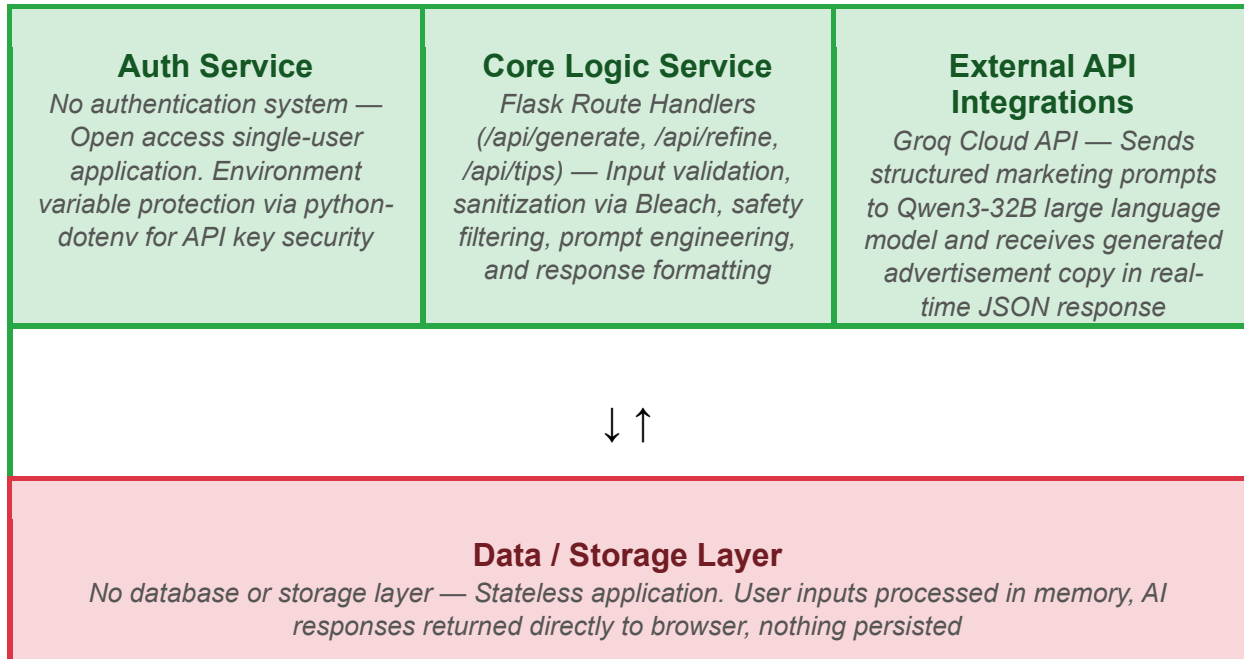
Solution Architecture

Date	26 June 2026
Team ID	SWTID-2026-9014
Project Name	AdForge AI: AI-Powered Advertisement Copy Generator
Maximum Marks	5 Marks

Solution Architecture Diagram:

AdForge AI follows a lightweight three-layer stateless architecture. The Presentation Layer consists of a browser-based web application built with HTML5, CSS3, and JavaScript where users fill in their campaign brief and receive AI-generated advertisement copy rendered as formatted Markdown. The Application Layer is a Python Flask server that receives user inputs via HTTP POST requests, validates and sanitizes all data using Bleach, applies content safety filtering, builds structured marketing prompts, and communicates with the external AI service. The External Integration Layer consists of the Groq Cloud API which processes the structured prompts through the Qwen3-32B large language model and returns platform-specific advertisement copy in real time. There is no database or data storage layer as the application is completely stateless — user inputs are processed in memory and results are returned directly to the browser without any data being saved or persisted.





Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	<i>Renders the user interface in the browser. Collects campaign brief form data from the user, sends it to the Flask backend via fetch() API calls, and displays AI-generated advertisement copy rendered as formatted Markdown</i>	<i>HTML5, CSS3, Vanilla JavaScript, Marked.js (Markdown renderer)</i>
API Gateway	<i>Acts as the entry point for all client requests. Routes HTTP POST requests to the correct Flask route handler based on the endpoint (/api/generate, /api/refine, /api/tips). Handles request parsing and response delivery</i>	<i>Python Flask 3.0.3, Werkzeug (development), Unicorn (production)</i>
Core Logic Service	<i>Contains all business logic of the application. Validates and sanitizes user inputs, checks for unsafe or restricted content, builds structured marketing prompts using advanced prompt engineering, calls the Groq API, and returns formatted responses</i>	<i>Python 3.8+, Bleach 6.1.0, python-dotenv 1.0.1, Custom prompt builders</i>
External API Integration	<i>Connects to the Groq Cloud API to access the Qwen3-32B large language model. Sends structured system and user prompts and receives AI-generated platform-specific advertisement copy in real-time</i>	<i>Groq Python SDK 0.9.0, Qwen3-32B LLM, httpx 0.27.0, REST API (JSON)</i>

<i>Safety & Validation Layer</i>	<i>Embedded within the Core Logic Service. Sanitizes all user inputs using Bleach to strip HTML tags, checks for banned keywords related to illegal or harmful content, and validates required fields before any API call is made</i>	<i>Python re module, Bleach 6.1.0, Custom keyword filter</i>
<i>Data / Storage Layer</i>	<i>No persistent storage used. All data flows in memory only — user inputs are received, processed, and responses returned within a single request-response cycle. No database, no session storage, no file storage</i>	<i>None — Stateless Architecture</i>